RELATÓRIO TÉCNICO E DE ARQUITETURA DE DADOS

Disciplina: Arquiteturas Disruptivas: IoT, Big Data e IA

Projeto: Pipeline de Dados com IoT e Docker

Tema: Ingestão, Armazenamento e Análise de Leituras de Temperatura de Dispositivos IoT

Aluno: Richard Humberto Pires de Jesus

Instituição: UniFECAF

Data: Outubro 2026

---

1. INTRODUÇÃO

A Internet das Coisas (Internet of Things — IoT) possibilita a conexão de dispositivos físicos capazes de coletar e transmitir dados continuamente. Esses dispositivos são utilizados em diferentes contextos, como monitoramento ambiental, automação industrial, controle de temperatura e acompanhamento de condições operacionais.

A utilização desses dispositivos gera grandes volumes de dados que precisam ser processados, armazenados e analisados de maneira organizada para que possam fornecer informações relevantes. Nesse cenário, os pipelines de dados desempenham um papel importante ao integrar diferentes tecnologias e automatizar etapas que vão desde a ingestão dos registros até sua disponibilização para análise.

Este projeto consiste no desenvolvimento de um pipeline de dados para processar leituras de temperatura provenientes de dispositivos IoT, utilizando Python para o processamento, PostgreSQL para o armazenamento, Docker para a execução isolada do banco de dados e Streamlit e Plotly para a construção de um dashboard interativo.

A solução foi desenvolvida com o propósito de integrar conceitos de engenharia de dados, armazenamento relacional e visualização analítica, demonstrando como diferentes ferramentas podem trabalhar em conjunto para transformar registros brutos em informações estruturadas.

Além da implementação técnica, o projeto contempla a organização dos arquivos, a documentação do ambiente e a utilização do Git e do GitHub para controle de versão e disponibilização do código-fonte.

2. OBJETIVOS

2.1. Objetivo geral

Desenvolver um pipeline de dados capaz de realizar a ingestão, o tratamento e o armazenamento de leituras de temperatura de dispositivos IoT em um banco de dados PostgreSQL, disponibilizando informações para análise por meio de um dashboard interativo.

2.2. Objetivos específicos

- Processar um conjunto de dados de temperatura disponibilizado em formato CSV.
- Utilizar Python para realizar a leitura e o tratamento dos registros.
- Integrar a aplicação ao PostgreSQL por meio de bibliotecas de conexão com bancos de dados.
- Executar o banco de dados em um contêiner Docker.
- Criar três views SQL para organizar informações relevantes para análise.
- Desenvolver visualizações para explorar as temperaturas e a distribuição temporal das leituras.
- Organizar os arquivos do projeto e disponibilizar o código por meio do GitHub.
- Documentar os procedimentos necessários para configuração e execução da solução.

3. ARQUITETURA E TECNOLOGIAS UTILIZADAS

A arquitetura do projeto foi organizada em componentes responsáveis por diferentes etapas do fluxo de dados. A aplicação Python realiza o processamento dos registros, o PostgreSQL armazena as informações, as views SQL disponibilizam consultas analíticas e o dashboard apresenta os resultados graficamente.

As principais tecnologias utilizadas são descritas a seguir.

3.1. Python

A linguagem Python foi utilizada no desenvolvimento dos scripts responsáveis pelo processamento dos dados e pela integração entre as diferentes partes da aplicação.

Sua utilização permite organizar as etapas de leitura, transformação e persistência dos registros, além de facilitar a comunicação com o banco de dados e a construção da aplicação de visualização.

3.2. Pandas

A biblioteca Pandas oferece recursos para leitura, organização e manipulação de dados tabulares.

No contexto do projeto, é utilizada para trabalhar com os registros provenientes do arquivo CSV e preparar os dados para as etapas posteriores do processamento.

3.3. PostgreSQL

O PostgreSQL é o sistema gerenciador de banco de dados relacional utilizado para armazenar os registros de temperatura.

O banco permite estruturar os dados em tabelas, executar consultas SQL e criar views para facilitar a análise das informações.

3.4. Docker

O Docker é utilizado para executar o PostgreSQL em um contêiner isolado. Essa abordagem facilita a configuração do ambiente e contribui para a reprodução da estrutura de banco de dados em diferentes máquinas.

O uso de contêineres também permite separar a execução do banco de dados das demais ferramentas utilizadas no desenvolvimento.

3.5. SQLAlchemy e Psycopg2

O SQLAlchemy é utilizado para gerenciar a comunicação entre a aplicação Python e o banco de dados por meio de uma camada de conexão e execução de consultas.

O Psycopg2 é o adaptador responsável pela comunicação entre Python e PostgreSQL no ambiente configurado.

Essas ferramentas permitem integrar o processamento dos dados às operações de armazenamento e consulta.

3.6. Streamlit e Plotly

O Streamlit é utilizado para desenvolver a interface do dashboard, permitindo apresentar os resultados das consultas em uma aplicação acessível pelo navegador.

O Plotly é utilizado para construir gráficos interativos, facilitando a visualização das médias de temperatura, da quantidade de leituras por hora e das temperaturas máximas e mínimas ao longo do tempo.

3.7. Git e GitHub

O Git é utilizado para o controle de versão dos arquivos do projeto, permitindo registrar alterações e acompanhar a evolução do desenvolvimento.

O GitHub é utilizado para hospedar o repositório remoto, disponibilizando o código-fonte e facilitando o acesso à implementação e à documentação.

4. ORGANIZAÇÃO DO PROJETO

A estrutura de diretórios foi organizada para separar os dados de origem, os scripts de aplicação e os arquivos relacionados à documentação e às consultas SQL.

A organização observada no ambiente de desenvolvimento inclui:

- `data/`: diretório destinado ao conjunto de dados utilizado no processamento.
- `src/`: diretório que contém os scripts Python da aplicação.
- `docs/`: diretório destinado aos documentos e materiais de apoio.
- `sql/`: diretório destinado às consultas SQL e às definições das views.
- `README.md`: arquivo de documentação com informações sobre o projeto e sua execução.
- `run.py`: arquivo localizado na raiz do projeto, destinado à execução conforme a implementação desenvolvida.

Na pasta `src/`, destacam-se os arquivos `pipeline.py` e `app.py`, associados às etapas de processamento e aplicação, respectivamente.

Essa organização facilita a manutenção do código, a localização dos componentes e a compreensão do funcionamento geral da solução.

5. INGESTÃO, TRATAMENTO E ARMAZENAMENTO DOS DADOS

5.1. Fonte dos dados

O conjunto de dados utilizado no projeto é o arquivo `data/IOT-temp.csv`, proveniente do dataset *Temperature Readings: IoT Devices*, disponibilizado na plataforma Kaggle.

O arquivo apresenta registros relacionados a leituras de temperatura, incluindo informações de identificação, localização ou ambiente, data, horário, temperatura e indicação de entrada ou saída, conforme as colunas presentes no conjunto de dados.

A base utilizada possui aproximadamente 97 mil registros, conforme o levantamento inicial realizado durante o desenvolvimento do projeto.

A utilização de dados reais permite explorar operações de processamento e análise sobre registros que representam situações de monitoramento por dispositivos IoT.

5.2. Ingestão dos dados

A ingestão corresponde à etapa de leitura dos registros disponibilizados na fonte de dados.

Nesse projeto, o arquivo CSV é utilizado como entrada para o processamento realizado em Python. Os registros são carregados para que possam ser organizados e preparados para armazenamento no PostgreSQL.

Essa etapa estabelece a ligação entre o conjunto de dados original e as demais etapas do pipeline.

5.3. Tratamento e transformação

Após a leitura do arquivo, os dados precisam apresentar tipos e formatos adequados para armazenamento e consulta.

O tratamento contempla a organização dos campos e a adequação dos tipos de dados utilizados pela aplicação. Atenção especial deve ser dada às informações temporais, uma vez que a interpretação incorreta de datas e horários pode comprometer consultas que agrupam registros por hora ou por dia.

Na conversão de datas, a função `TO_TIMESTAMP`, disponível no PostgreSQL, pode ser utilizada para interpretar valores textuais conforme o formato de origem. Quando aplicável, o padrão `DD-MM-YYYY` deve corresponder à estrutura real dos valores recebidos.

O tratamento dos registros temporais é importante para permitir a construção de análises cronológicas consistentes e a utilização das funções de agregação nas views SQL.

5.4. Armazenamento no banco de dados

Após o processamento, os registros são encaminhados ao PostgreSQL, responsável pela persistência das informações.

O banco de dados é executado em um contêiner Docker, permitindo manter o ambiente de armazenamento separado dos demais componentes da aplicação.

A tabela principal utilizada nas consultas do projeto é identificada como `temperature_readings`, conforme a estrutura considerada na implementação inicial.

O armazenamento relacional centraliza os registros e possibilita a realização de consultas SQL para agrupamento, contagem e cálculo de indicadores estatísticos.

5.5. Fluxo geral do pipeline

O fluxo lógico do projeto pode ser representado pelas seguintes etapas:

Arquivo CSV → Leitura e tratamento em Python → Armazenamento no PostgreSQL → Consultas por meio de views SQL → Visualização no dashboard Streamlit.

Cada componente possui uma função específica nesse fluxo. O arquivo CSV representa a fonte dos dados; Python realiza o processamento; PostgreSQL mantém os registros armazenados; as views SQL organizam as consultas analíticas; e Streamlit e Plotly disponibilizam os resultados visualmente.

Essa divisão facilita a compreensão da arquitetura e permite identificar as responsabilidades de cada tecnologia utilizada.

6. VIEWS ANALÍTICAS IMPLEMENTADAS EM SQL

As views SQL foram definidas para organizar consultas recorrentes e facilitar a análise dos registros armazenados no PostgreSQL.

Uma view corresponde a uma consulta SQL armazenada no banco de dados, que pode ser utilizada como fonte para outras consultas e aplicações. Nesse projeto, as três views têm como finalidade explorar diferentes aspectos das leituras de temperatura.

6.1. View `avg_temp_por_dispositivo`

A view `avg_temp_por_dispositivo` tem como objetivo calcular a temperatura média agrupada pelo identificador utilizado na análise dos dispositivos ou ambientes monitorados.

O cálculo da média pode ser realizado com a função de agregação `AVG()`, combinada com o agrupamento dos registros por meio de `GROUP BY`.

Objetivo analítico: permitir a comparação das temperaturas médias entre as unidades identificadas na base.

Essa consulta pode contribuir para a compreensão do comportamento térmico dos ambientes monitorados e para a identificação de diferenças entre os grupos analisados.

A interpretação dos resultados deve considerar o significado real dos identificadores presentes na base. Caso a coluna represente ambientes ou salas, a análise deverá ser descrita dessa forma; caso represente dispositivos, deverá refletir essa característica.

Visualização associada: gráfico de barras da média de temperatura por dispositivo ou unidade de identificação.



Figura 1 — Média de temperatura por dispositivo ou unidade identificada na base.

Fonte: elaborado pelo autor (2026).

6.2. View `leituras_por_hora`

A view `leituras_por_hora` tem como objetivo contabilizar a quantidade de registros em cada hora do dia.

A consulta utiliza operações de extração da hora e contagem dos registros, agrupando os resultados conforme o intervalo temporal correspondente.

Objetivo analítico: compreender a distribuição das leituras ao longo do dia e identificar os horários que concentram maior ou menor quantidade de registros.

Essa análise pode ajudar a observar padrões de distribuição temporal e a avaliar a concentração dos registros em determinados períodos.

É importante destacar que a quantidade de leituras não representa diretamente a temperatura registrada. Dessa forma, horários com maior volume de registros não devem ser interpretados automaticamente como períodos mais quentes.

Visualização associada: gráfico de linhas que apresenta a quantidade de leituras em função da hora do dia.



Figura 2 — Distribuição das leituras por hora do dia.

Fonte: elaborado pelo autor (2026).

6.3. View `temp_max_min_por_dia`

A view `temp_max_min_por_dia` tem como objetivo calcular as temperaturas máxima e mínima registradas em cada dia.

Para isso, podem ser utilizadas as funções de agregação `MAX()` e `MIN()`, com agrupamento dos registros pela data correspondente.

Objetivo analítico: acompanhar os extremos de temperatura ao longo do período analisado e observar a variação diária dos valores registrados.

A comparação entre as temperaturas máximas e mínimas permite avaliar a amplitude térmica diária, definida pela diferença entre o maior e o menor valor registrado em um determinado dia.

Essa informação pode ser útil em aplicações de monitoramento ambiental, controle térmico e investigação de variações que mereçam análise adicional.

Visualização associada: gráfico de linhas com as temperaturas máximas e mínimas agrupadas por data.

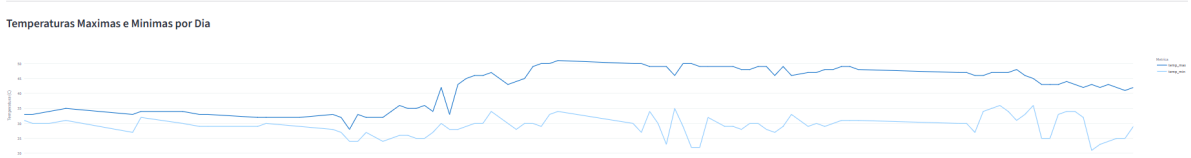


Figura 3 — Temperaturas máximas e mínimas registradas por dia.

Fonte: elaborado pelo autor (2026).

7. DESENVOLVIMENTO DO DASHBOARD INTERATIVO

O dashboard foi desenvolvido com Streamlit e Plotly, com a finalidade de apresentar os resultados das consultas analíticas em uma interface gráfica acessível pelo navegador.

A aplicação utiliza as informações disponibilizadas pelas views SQL como fonte para a construção das visualizações. Essa abordagem permite manter a lógica de consulta no banco de dados e utilizar a aplicação para apresentar os resultados de forma visual.

O painel apresenta três áreas principais de análise.

A primeira área exibe a média de temperatura por dispositivo ou unidade identificada na base, permitindo comparar os valores médios calculados.

A segunda área apresenta a quantidade de leituras por hora do dia. Essa visualização facilita a observação da distribuição dos registros ao longo das 24 horas.

A terceira área apresenta as temperaturas máximas e mínimas por dia. O gráfico permite acompanhar o comportamento dos extremos de temperatura ao longo do período representado.

A captura de tela do dashboard demonstra que a interface de visualização foi aberta no navegador e que os três gráficos estão apresentados na aplicação.

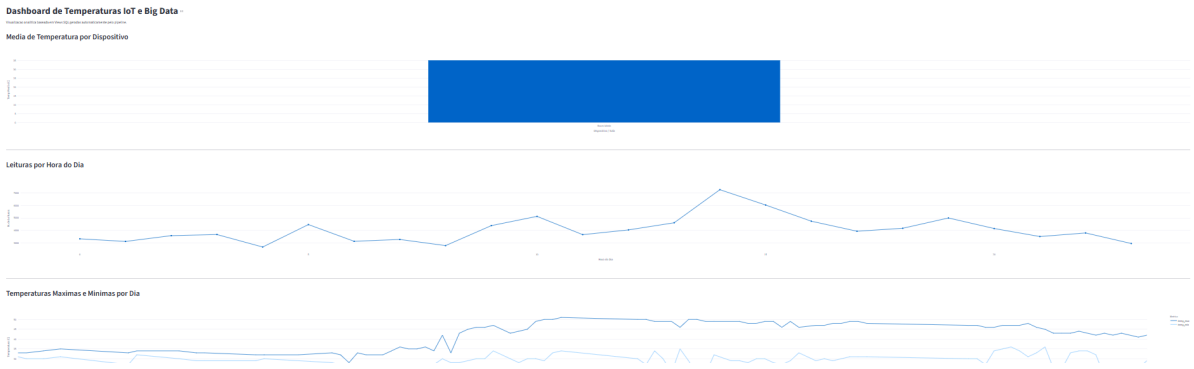


Figura 4 — Dashboard de análise de temperaturas de dispositivos IoT.

Fonte: elaborado pelo autor (2026).

A visualização gráfica contribui para tornar os resultados das consultas mais acessíveis, facilitando a interpretação das informações sem a necessidade de interação direta com comandos SQL.

8. RESULTADOS E EVIDÊNCIAS DA IMPLEMENTAÇÃO

A execução do projeto permitiu apresentar um dashboard com três visualizações destinadas à análise dos dados de temperatura. As imagens registradas durante o desenvolvimento também documentam a organização dos arquivos e o trabalho realizado no ambiente de programação.

8.1. Resultados visuais

O primeiro gráfico apresenta a média de temperatura associada à categoria identificada como `Room Admin` na captura registrada. A imagem evidencia uma categoria exibida no gráfico, mas não é suficiente para determinar se esse resultado corresponde a toda a base ou a um agrupamento específico definido na consulta.

O segundo gráfico apresenta a quantidade de leituras ao longo das horas do dia. Visualmente, observa-se que o volume de registros varia conforme o horário, com um pico aparente próximo às 14 horas na captura apresentada.

O terceiro gráfico apresenta as temperaturas máximas e mínimas ao longo do período exibido. As duas linhas permitem observar diferenças entre os extremos diários e mudanças no comportamento dos registros ao longo do tempo.

Essas observações são de caráter visual e preliminar. A confirmação dos valores exatos e a elaboração de conclusões quantitativas dependem da consulta aos resultados numéricos das views e da verificação da estrutura dos dados.

8.2. Evidência do ambiente de desenvolvimento

A estrutura apresentada no Visual Studio Code demonstra a organização do projeto em diretórios destinados aos dados, aos scripts Python, à documentação e aos arquivos SQL.

Também é possível observar a execução do comando `git push origin main` no terminal, com mensagens indicando que os objetos foram enviados ao repositório remoto e que a referência local `main` foi atualizada no destino remoto.

Essa evidência documenta o uso do Git para disponibilização das alterações do projeto. O envio bem-sucedido dos arquivos, por si só, não comprova que todas as etapas do pipeline foram executadas ou testadas.

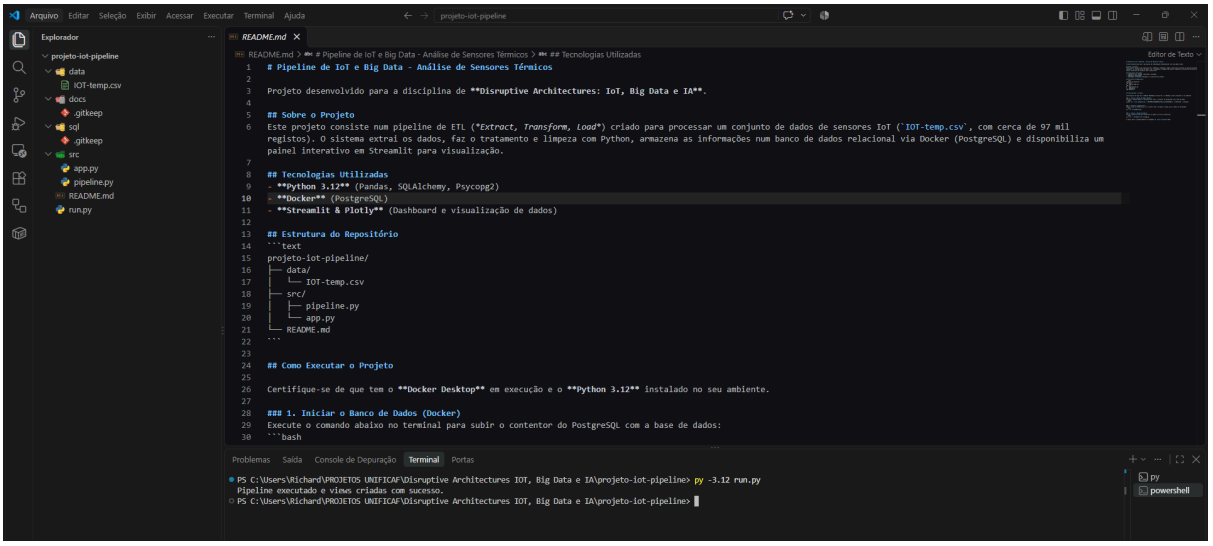


Figura 5 — Estrutura de arquivos do projeto e envio das alterações ao GitHub.

Fonte: elaborado pelo autor (2026).

8.3. Síntese dos resultados

Com base nas evidências disponíveis, foi possível demonstrar a apresentação das três visualizações do dashboard e documentar a organização do ambiente de desenvolvimento e o envio das alterações ao repositório remoto.

Para complementar a validação técnica, recomenda-se registrar também a confirmação da quantidade de registros armazenados, a execução das três views SQL e a reprodução da aplicação a partir das instruções de documentação.

9. POSSÍVEIS APLICAÇÕES PRÁTICAS E TRABALHOS FUTUROS

A arquitetura desenvolvida apresenta possibilidades de aplicação em cenários que necessitam organizar e analisar dados de temperatura provenientes de dispositivos conectados.

Em ambientes industriais, por exemplo, análises de temperaturas máximas e mínimas podem contribuir para o acompanhamento das condições operacionais de equipamentos e espaços físicos.

Em ambientes de armazenamento, como depósitos e câmaras térmicas, a análise temporal pode apoiar o acompanhamento das variações de temperatura e a investigação de períodos que demandem atenção.

Em aplicações de monitoramento ambiental, a comparação de valores médios entre locais ou unidades identificadas pode contribuir para a compreensão das diferenças observadas nos registros.

Esses exemplos representam possibilidades de utilização da arquitetura, não resultados de aplicações reais já comprovadas pelo projeto.

Como trabalhos futuros, podem ser consideradas as seguintes melhorias:

- Implementação de verificações automatizadas para identificar registros ausentes, duplicados ou inconsistentes.
- Inclusão de métricas adicionais, como mediana, desvio-padrão e amplitude térmica.
- Aprimoramento dos filtros do dashboard para facilitar a exploração por período e unidade monitorada.
- Implementação de mecanismos de atualização periódica dos dados.
- Registro de logs e tratamento de erros durante a ingestão.
- Organização de parâmetros de conexão e credenciais por meio de variáveis de ambiente.
- Ampliação dos testes para validar a integridade dos dados armazenados e a reprodutibilidade do pipeline.

Essas melhorias podem contribuir para tornar a solução mais robusta, facilitando sua manutenção e sua adaptação a novos conjuntos de dados.

10. REPRODUTIBILIDADE E DOCUMENTAÇÃO

A organização do código e a utilização do GitHub são importantes para facilitar o acesso ao projeto e permitir que outras pessoas compreendam os procedimentos utilizados no desenvolvimento.

O repositório deve conter os scripts Python, as definições das views SQL, as instruções de configuração do PostgreSQL com Docker e a documentação necessária para executar a aplicação.

O arquivo `README.md` deve descrever o objetivo do projeto, as tecnologias utilizadas, as dependências necessárias, os comandos de execução e a forma de obtenção do conjunto de dados original.

A reprodução do ambiente deve considerar a instalação das ferramentas necessárias, a configuração do banco de dados, a disponibilidade do arquivo CSV e os parâmetros utilizados pela aplicação para estabelecer a conexão com o PostgreSQL.

Também é importante evitar a publicação de senhas ou outras informações sensíveis no repositório remoto.

A documentação adequada contribui para a transparência do processo de desenvolvimento e facilita a avaliação e a continuidade do trabalho.

11. CONCLUSÃO

O projeto teve como propósito desenvolver um pipeline de dados voltado ao processamento e à análise de leituras de temperatura provenientes de dispositivos IoT, integrando Python, PostgreSQL, Docker, SQL e ferramentas de visualização.

A arquitetura proposta organiza o fluxo desde a leitura dos dados em formato CSV até sua apresentação em um dashboard interativo. O uso do PostgreSQL possibilita o armazenamento estruturado e a execução de consultas analíticas, enquanto as views SQL organizam as informações para a construção dos gráficos.

As três visualizações desenvolvidas abordam diferentes dimensões dos dados: a média de temperatura por unidade identificada, a distribuição das leituras por hora e as temperaturas máximas e mínimas por dia. Em conjunto, essas análises permitem explorar os registros e identificar padrões que podem ser investigados de forma mais aprofundada.

As evidências registradas demonstram a apresentação do dashboard no navegador e a organização dos arquivos no ambiente de desenvolvimento, incluindo o envio das alterações ao repositório remoto.

Como aprendizado, o projeto permite aplicar conceitos de integração de sistemas, processamento de dados, armazenamento relacional, consultas SQL, visualização analítica e controle de versão em uma solução integrada.

Conclui-se que a proposta oferece uma base para a construção de aplicações de análise de dados de IoT e demonstra a importância da integração entre diferentes tecnologias para transformar registros brutos em informações acessíveis. A validação integral da solução deve considerar os testes do pipeline, a consistência dos dados armazenados e a possibilidade de reproduzir o ambiente a partir da documentação disponibilizada.

12. REFERÊNCIAS

KAGGLE. *Temperature Readings: IoT Devices*. Disponível em: https://www.kaggle.com/datasets/atulanandjha/temperature-readings-iot-devices. Acesso em: 11 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*. Disponível em: https://www.postgresql.org/docs/. Acesso em: 11 out. 2026..

PYTHON SOFTWARE FOUNDATION. *Python Documentation*. Disponível em: https://docs.python.org/3/. Acesso em: 11 out. 2026..

DOCKER INC. *Docker Documentation*. Disponível em: https://docs.docker.com/. Acesso em: 11 out. 2026.

STREAMLIT. *Streamlit Documentation*. Disponível em: https://docs.streamlit.io/. Acesso em: 11 out. 2026..

PLOTLY. *Plotly Python Graphing Library*. Disponível em: https://plotly.com/python/. Acesso em:11 out. 2026.

GITHUB. *GitHub Documentation*. Disponível em: https://docs.github.com/. Acesso em: 11 out. 2026.